

Traitement d'images

Vision par ordinateur

4. Réseaux de neurones profonds

Julie Halbout et Michèle Gouiffès

julie.halbout@lisn.fr

université
PARIS-SACLAY



- 1 Généralités
- 2 Réseaux de neurones
- 3 Exemples d'algorithmes d'apprentissage profond
- 4 Exercice 1 : entraînement d'un réseau MLP
- 5 Exercice 2 : entraînement d'un réseau convolutif
- 6 Exercice 3 : utilisation d'un modèle entraîné

Intro : apprentissage automatique pour la vision

Exemples d'applications

Classification



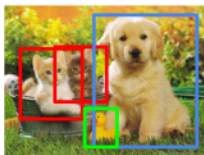
CAT

**Classification
+ Localization**



CAT

Object Detection



CAT, DOG, DUCK

**Instance
Segmentation**

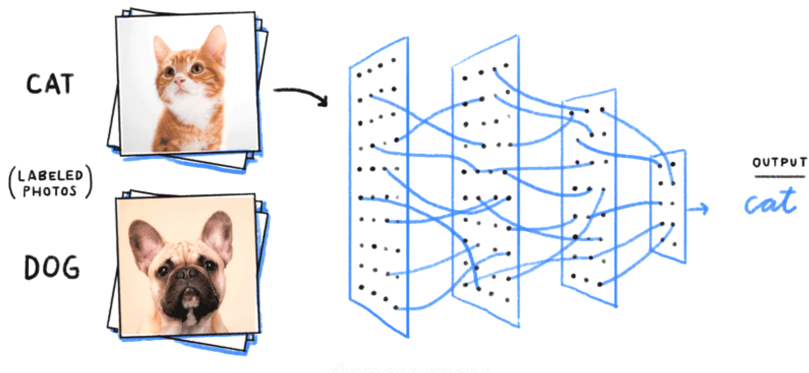


CAT, DOG, DUCK

Single object

Multiple objects

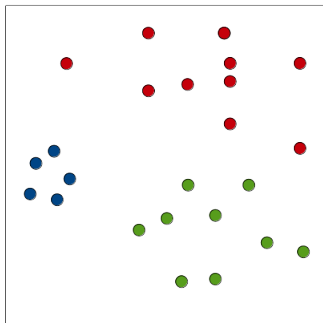
Intro : apprentissage automatique pour la vision



La plupart des méthodes requièrent des masses de données, éventuellement annotées, par exemple des milliers d'images de chats, annotées « chat » des milliers d'exemples de chiens annotées « chien »

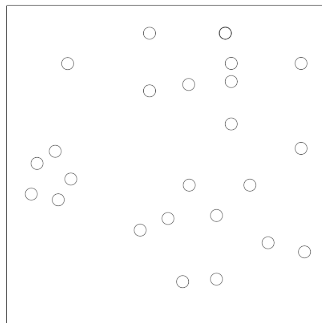
Généralités : apprentissage supervisé et non supervisé

Supervisé



Nécessite données X
+ variables à prédire y
(labels représentés ici
par des couleurs)

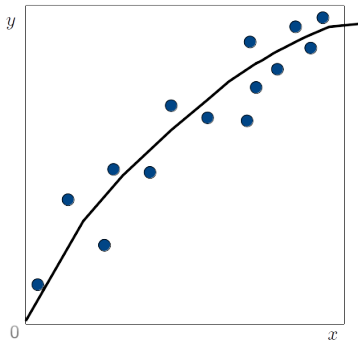
Non supervisé



Nécessite données
d'entraînement X
sans label, sans annotations
→ Clustering k-means, regroupements

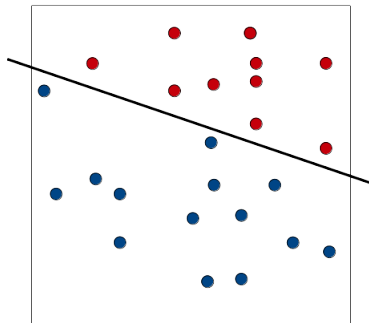
Généralités : Régression et classification

Régression



Prédire
une variable quantitative

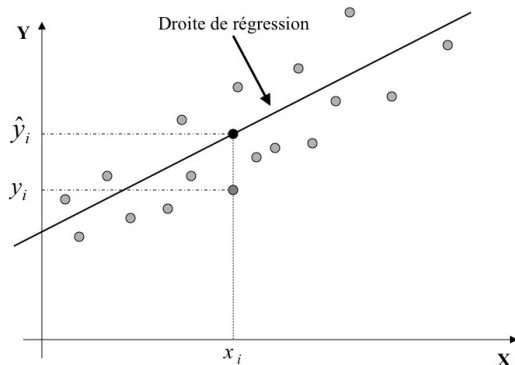
Classification



Prédire
une classe (qualitative, discrète)

Généralités : minimiser une fonction de coût

Exemple simple de fonction de coût avec une régression linéaire



x_i, y_i : données

$\hat{y}_i$: prédiction associée à une nouvelle donnée d'entrée x_i

On suppose que la prédiction peut s'écrire :

$$\hat{y}_i = w_1 x_i + w_0$$

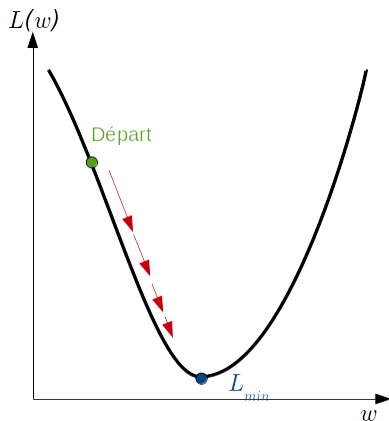
Les paramètres w_1, w_0 sont estimés en minimisant une **fonction de coût** :

$$L(w) = \frac{1}{m} \sum_{i=1}^m (\hat{y}_i - y_i)^2$$

$$L(w) = \frac{1}{m} \sum_{i=1}^m (w_1 x_i + w_0 - y_i)^2$$

Généralités : descente de gradient

La fonction de coût est minimisée de façon itérative en utilisant une descente de gradient.



Gradient :

vecteur de dérivées partielles $\frac{\partial L}{\partial w}$

Descente de gradient :

$$w_i = w_i - \alpha \frac{\partial L}{\partial w_i}$$

Exemple :

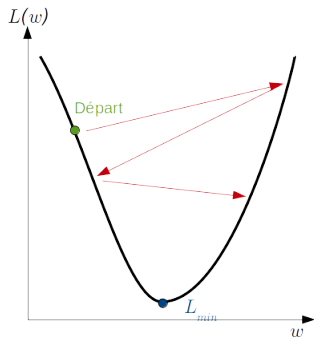
$$L = \frac{1}{m} \sum_{i=1}^m (w_1 x_i + w_0 - y_i)^2$$

$$\frac{\partial L}{\partial w_0} = \frac{2}{m} \sum_{i=1}^m (\hat{y}_i - y_i)$$

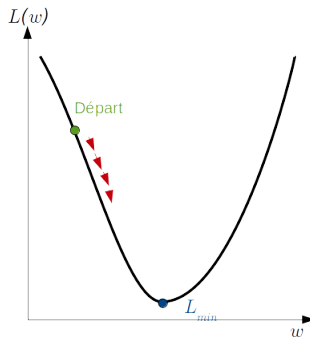
$$\frac{\partial L}{\partial w_1} = \frac{2}{m} \sum_{i=1}^m x_i (\hat{y}_i - y_i)$$

Généralités : descente de gradient

Coefficient d'apprentissage α



Coefficient trop élevé
→ mauvaise convergence



Coefficient trop petit
→ convergence lente

Généralités : descente de gradient

La descente de gradient nécessite le calcul des dérivées partielles sur m données :

$$\text{Exemple précédent : } \frac{\partial L}{\partial w_0} = \frac{2}{m} \sum_{i=1}^m (\hat{y}_i - y_i)$$

Plusieurs solutions :

- Full Batch Gradient Descent (FBGD) : calcul du gradient sur tous les échantillons d'un coup, ce qui peut être **très long** si m est grand
- Stochastic Gradient descent (SGD) : échantillon par échantillon, **convergence chaotique**
- Mini Batch Gradient Descent : k échantillons à la fois (50 par exemple), **bon compromis**

Epoch : passage de l'ensemble des données, ce qui correspond à une étape dans la descente de gradient.

Généralités : entraînement

L'entraînement consiste à estimer les paramètres w itérativement par **descente de gradient**.

La convergence dépend des **hyperparamètres** : le coefficient d'entraînement α , le nombre d'épochs, la taille du batch.

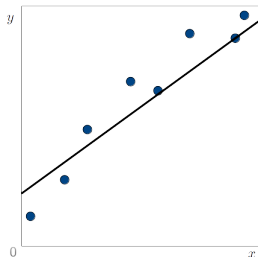
Évaluation du modèle d'apprentissage : 3 étapes

- 1 **Entraînement** : sur le jeu d'entraînement
- 2 **Validation** : les hyperparamètres sont choisis pour optimiser la performance, en utilisant un jeu de données de validation
- 3 **Test** : on ne modifie plus les paramètres et la performance (*accuracy*) est évaluée sur le jeu de test

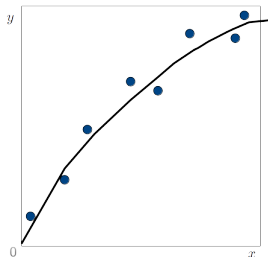


Cela permet de généraliser tout en vérifiant que le modèle fonctionne bien sur de nouvelles données, que le modèle n'a jamais vu.

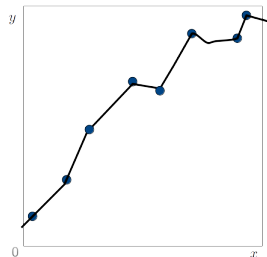
Généralités : sur-apprentissage et sous-apprentissage



Sous-apprentissage

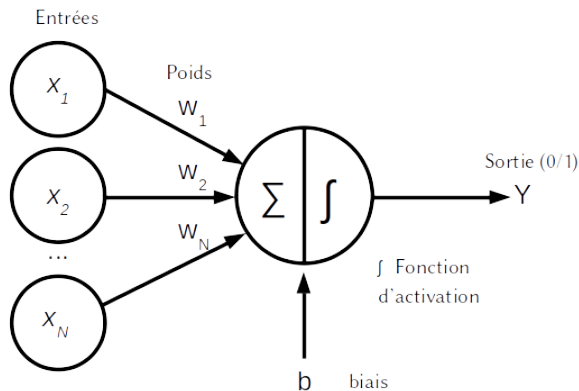


ok



Sur-apprentissage
colle trop au bruit
du jeu de donnée

Réseaux de neurones : le perceptron

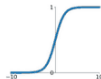


Fonctions d'activations

Activation Functions

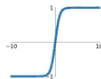
Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}$$



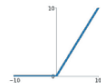
tanh

$$\tanh(x)$$



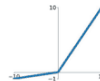
ReLU

$$\max(0, x)$$



Leaky ReLU

$$\max(0.1x, x)$$

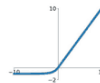


Maxout

$$\max(w_1^T x + b_1, w_2^T x + b_2)$$

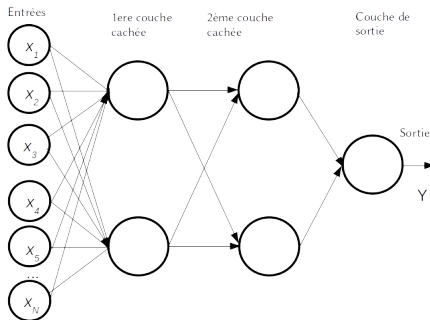
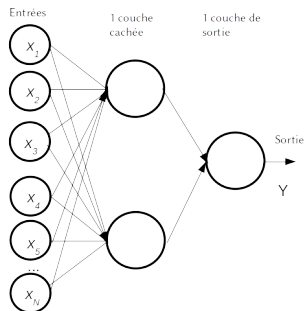
ELU

$$\begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$

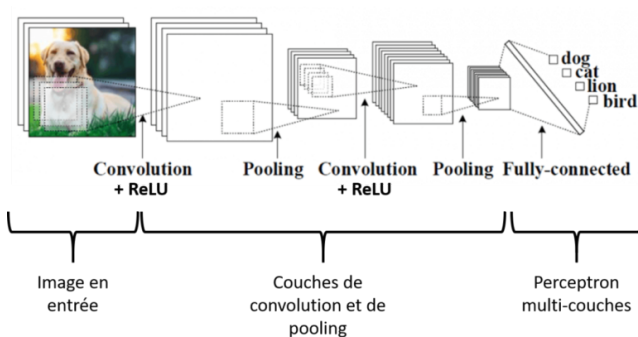


Source

Réseaux de neurones : le perceptron multi-couche



Réseaux de neurones : le réseau convolutionnel



Source